

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN

Nhóm sinh viên:

Vòng Sau Hùng (22120118)

Phan Tấn Phát (22120264)

Nguyễn Quốc Tường (22120413)

Huỳnh Trần Ty (22120418)

Nguyễn Trường Vũ (22120441)

Bùi Đoàn Thuý Vy (22120448)

XÂY DỰNG HỆ THỐNG HỖ TRỢ TÌM NHÀ TRỢ TÍCH
HỢP AI

ĐỒ ÁN TỐT NGHIỆP CỬ NHÂN
CHƯƠNG TRÌNH CHÍNH QUY

Tp. Hồ Chí Minh, tháng 07/2026

**TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN**

**Nhóm sinh viên thực hiện:
Vòng Sau Hùng (22120118)
Phan Tấn Phát (22120264)
Nguyễn Quốc Tường (22120413)
Huỳnh Trần Ty (22120418)
Nguyễn Trường Vũ (22120441)
Bùi Đoàn Thuý Vy (22120448)**

**XÂY DỰNG HỆ THỐNG HỖ TRỢ TÌM NHÀ TRỢ TÍCH
HỢP AI**

**ĐỒ ÁN TỐT NGHIỆP CỬ NHÂN
CHƯƠNG TRÌNH CHÍNH QUY**

NGƯỜI HƯỚNG DẪN

ThS. Phạm Minh Tuấn

Tp. Hồ Chí Minh, tháng 07/2026

Lời cam đoan

Chúng tôi xin cam đoan đây là công trình nghiên cứu của chúng tôi dưới sự hướng dẫn của giảng viên hướng dẫn ThS. Phạm Minh Tuấn. Chúng tôi xin chịu trách nhiệm về tính chính xác của các thông tin và kết quả nghiên cứu trong báo cáo này. Nếu có bất kỳ sai sót nào, chúng tôi xin hoàn toàn chịu trách nhiệm trước khoa Công nghệ thông tin, trường Đại học Khoa học Tự nhiên và các cơ quan liên quan.

Lời cảm ơn

Nhóm sinh viên chúng em xin được gửi lời cảm ơn chân thành và sâu sắc đến:

Ban Giám hiệu Trường Đại học Khoa học Tự nhiên – Đại học Quốc gia Thành phố Hồ Chí Minh, cùng Ban Chủ nhiệm Khoa Công nghệ Thông tin đã tạo điều kiện thuận lợi, cung cấp môi trường học tập và nghiên cứu tích cực để chúng em có thể thực hiện và hoàn thành đồ án tốt nghiệp với chủ đề “Xây dựng hệ thống hỗ trợ tìm nhà trọ tích hợp AI”.

Chúng em xin bày tỏ lòng biết ơn sâu sắc đến ThS. Phạm Minh Tuấn – là giảng viên hướng dẫn, người đã tận tình đồng hành, theo sát tiến độ của nhóm trong suốt quá trình thực hiện đồ án. Thầy không chỉ dành thời gian góp ý chuyên môn một cách cụ thể, xác đáng, mà còn truyền cảm hứng và định hướng cho nhóm áp dụng những công nghệ hiện đại và phù hợp, giúp sản phẩm đạt được tính ứng dụng thực tiễn cao.

Trong quá trình thực hiện đề tài, mặc dù đã rất nỗ lực nhưng chắc chắn không thể tránh khỏi những thiếu sót nhất định. Chúng em rất mong nhận được sự góp ý quý báu từ quý thầy cô để có thể hoàn thiện sản phẩm và nâng cao năng lực chuyên môn hơn nữa.

Những kiến thức, kỹ năng và trải nghiệm mà chúng em thu nhận được trong suốt quá trình làm đồ án sẽ là hành trang quý báu cho sự phát triển nghề nghiệp trong tương lai.

Một lần nữa, nhóm xin trân trọng cảm ơn tất cả quý thầy cô và những cá nhân đã hỗ trợ, đồng hành cùng nhóm trong chặng đường thực hiện đồ án tốt nghiệp này.

Đề cương chi tiết

Mục lục

Lời cam đoan	i
Lời cảm ơn	ii
Đề cương	iii
Mục lục	iv
Danh sách hình	x
Danh sách bảng	xi
Danh mục từ viết tắt và thuật ngữ	xi
Tóm tắt	xv
1 Giới thiệu đề tài	1
1.1 Giới thiệu	1
1.2 Mục tiêu đề tài	1
1.3 Phạm vi đề tài	1
1.4 Phương pháp nghiên cứu	1
1.5 Ý nghĩa khoa học và thực tiễn	1
1.6 Cấu trúc báo cáo	1
2 Tổng quan	2
2.1 Tổng quan về hệ thống thuê nhà trọ trực tuyến	2

2.1.1	Khái niệm nhà trọ và thị trường cho thuê nhà trọ .	2
2.1.2	Các mô hình hệ thống thuê nhà hiện nay	2
2.2	Khảo sát các nền tảng hiện có	2
2.2.1	Batdongsan.com.vn	3
2.2.2	Phongtro123	3
2.2.3	Nhà Tốt	3
2.2.4	Zillow	4
2.2.5	So sánh ưu nhược điểm	4
2.3	Cơ sở lý thuyết về trí tuệ nhân tạo	5
2.4	Khoảng trống và cơ hội ứng dụng AI trong lĩnh vực cho thuê nhà trọ	5
2.5	Kết luận chương	5
3	Phân tích và đặc tả yêu cầu hệ thống	6
3.1	Mô tả bài toán	7
3.2	Phân tích nghiệp vụ	7
3.2.1	Người thuê trọ	7
3.2.2	Chủ nhà trọ	7
3.2.3	Quản trị viên	7
3.3	Yêu cầu chức năng	7
3.3.1	Đăng ký, đăng nhập	7
3.3.2	Quản lý hồ sơ người dùng	7
3.3.3	Đăng tin cho thuê phòng	7
3.3.4	Tìm kiếm phòng trọ	7
3.3.5	Đặt lịch xem phòng	7
3.3.6	Quản lý hợp đồng thuê	7
3.3.7	Đánh giá và phản hồi	7
3.3.8	Chat hỗ trợ	7
3.4	Yêu cầu phi chức năng	7
3.4.1	Hiệu năng	7
3.4.2	Bảo mật	7

3.4.3	Khả năng mở rộng	7
3.4.4	Tính khả dụng	7
3.5	Phân tích AI hỗ trợ hệ thống	7
3.5.1	Chatbot tư vấn tìm phòng	7
3.5.2	Gợi ý phòng trọ thông minh	7
3.5.3	Phân tích nhu cầu người dùng	7
3.5.4	Tóm tắt thông tin phòng trọ bằng AI	7
3.6	Đặc tả Use Case	7
4	Thiết kế hệ thống	8
4.1	Kiến trúc tổng thể hệ thống	8
4.1.1	Mô hình kiến trúc hệ thống	8
4.1.2	Khái quát các thành phần bên trong hệ thống . . .	8
4.2	Thiết kế nền tảng Web	8
4.2.1	Tổ chức mã nguồn	9
4.2.2	Cơ chế kết nối và tích hợp	11
4.2.3	Hỗ trợ đa ngôn ngữ	13
4.2.4	Ưu điểm	14
4.3	Thiết kế hệ thống Backend	15
4.3.1	Kiến trúc Modular Monolith	16
4.3.2	Các module nghiệp vụ	18
4.3.3	Tổ chức phân tầng bên trong module	18
4.3.4	Luồng dữ liệu	20
4.3.5	Ưu điểm	21
4.4	Thiết kế kiến trúc AI	22
4.4.1	Luồng xử lý AI Chatbot	22
4.4.2	Luồng gợi ý phòng trọ	22
4.4.3	Tích hợp API AI	22
4.4.4	Kiến trúc tích hợp AI	22
4.5	Thiết kế cơ sở dữ liệu	22
4.5.1	Giới thiệu tổng quan	22

4.5.2	Sơ đồ thiết kế cơ sở dữ liệu của hệ thống chính . . .	22
4.5.3	Mô tả chi tiết các bảng	22
4.6	Thiết kế giao diện	22
4.6.1	Giao diện người thuê	22
4.6.2	Giao diện chủ trọ	24
4.6.3	Giao diện quản trị	24
4.6.4	Giao diện chatbot AI	24
5	Xây dựng và triển khai hệ thống	25
5.1	Môi trường phát triển	25
5.2	Các giai đoạn của đề tài	25
5.2.1	Xây dựng phiên bản MVP (Tháng 12/2025 – 02/2026)	25
5.2.2	Hoàn thiện các chức năng nghiệp vụ (Tháng 02/2025 – 04/2026)	25
5.2.3	Tích hợp AI nâng cao, thanh toán và hoàn thiện hệ thống (Tháng 05/2026 – 07/2026)	25
5.3	Quy trình làm việc nhóm	25
5.3.1	Phân công vai trò	25
5.3.2	Phương pháp làm việc nhóm	25
5.3.3	Phân chia nhiệm vụ và theo dõi tiến độ	27
5.4	Công nghệ sử dụng	30
5.4.1	Frontend	30
5.4.2	Backend	30
5.4.3	Database	30
5.4.4	AI Service	30
5.5	Xây dựng các module chức năng	30
5.5.1	Quản lý người dùng	30
5.5.2	Quản lý phòng trọ	30
5.5.3	Tìm kiếm và lọc	30
5.5.4	Đặt lịch xem phòng	30
5.5.5	Hệ thống đánh giá	30

5.6	Xây dựng module AI	30
5.6.1	AI Agent hỗ trợ tìm kiếm nhà trọ	30
5.6.2	AI đánh giá giá bất động sản	30
5.6.3	AI tạo nội dung bài đăng	30
5.6.4	Hệ thống gợi ý cá nhân hóa	30
5.6.5	AI kiểm duyệt nội dung bài đăng	30
5.7	Triển khai hệ thống	30
5.7.1	R2	30
5.7.2	MySQL	30
5.7.3	LiteLLM & Langfuse	30
5.7.4	Cổng thanh toán	30
5.7.5	CI/CD Pipeline	30
5.7.6	Triển khai ứng dụng	30
5.8	Các khó khăn và giải pháp trong quá trình xây dựng hệ thống	30
5.9	Kết quả giao diện dành cho người thuê	30
5.9.1	Trang chủ và khám phá tin đăng	31
5.9.2	Tìm kiếm và lọc tin đăng	31
5.9.3	Trang chi tiết tin đăng	32
5.9.4	Bản đồ tin đăng	32
5.9.5	So sánh tin đăng	32
5.9.6	Lưu tin và theo dõi người đăng	33
5.9.7	Trợ lý hội thoại AI trên giao diện	33
5.9.8	Đăng nhập và xác thực	33
5.9.9	Tin tức và thông báo	34
5.9.10	Trải nghiệm đa thiết bị và đa ngôn ngữ	34
5.10	Kết quả giao diện dành cho quản trị viên	34
5.10.1	Thống kê & Giám sát Hệ thống	34
5.10.2	Quản lý Người dùng & Phân quyền	34
5.10.3	Quản lý tin đăng và kiểm duyệt nội dung	34
5.10.4	Quản lý doanh thu và thanh toán	34

6	Kiểm thử và đánh giá hệ thống	35
6.1	Kế hoạch kiểm thử	35
6.2	Kiểm thử chức năng	35
6.3	Kiểm thử API	35
6.4	Kiểm thử bảo mật	35
6.5	Kiểm thử hiệu năng	35
6.6	Đánh giá chức năng AI	35
6.6.1	Độ chính xác của gợi ý phòng	35
6.6.2	Chất lượng phản hồi chatbot	35
6.6.3	Khảo sát mức độ hài lòng người dùng	35
6.7	So sánh với các hệ thống hiện có	35
7	Kết luận và hướng phát triển	36
7.1	Kết quả đạt được	36
7.2	Kiến thức và kinh nghiệm thu nhận được	36
7.2.1	Về mặt kỹ thuật	36
7.2.2	Về kỹ năng làm việc nhóm và lập kế hoạch	36
7.2.3	Một số kiến thức và kinh nghiệm khác	36
7.3	Hạn chế của hệ thống	36
7.4	Hướng phát triển	36
	Tài liệu tham khảo	37

Danh sách hình

4.1	Sơ đồ tổ chức mã nguồn ứng dụng web (webapp)	11
4.2	Luồng dữ liệu giữa giao diện, hook, tầng dịch vụ và Backend	13
4.3	Kiến trúc phân lớp của hệ thống Backend (Modular Monolith)	16
4.4	Các dịch vụ bên ngoài mà Backend tích hợp	17
4.5	Các module nghiệp vụ của Backend gom theo cụm chức năng	18
4.6	Cấu trúc gói <code>com.smartrent</code> và cách phân tầng	20
4.7	Luồng xử lý một yêu cầu có xác thực ở Backend	21

Danh sách bảng

2.1	So sánh các nền tảng tìm kiếm bất động sản cho thuê hiện có	4
4.1	Cấu trúc thư mục mã nguồn của ứng dụng web	9
4.2	Cấu trúc gói mã nguồn của Backend (<code>com.smartrent</code>) . .	19

Danh mục từ viết tắt và thuật ngữ

Từ viết tắt	Từ đầy đủ	Giải thích
AI	Artificial Intelligence	Lĩnh vực khoa học máy tính nghiên cứu các hệ thống có khả năng thực hiện tác vụ đòi hỏi trí thông minh con người.
LLM	Large Language Model	Mô hình học sâu với hàng tỷ tham số, có khả năng hiểu và tạo sinh ngôn ngữ tự nhiên.
RAG	Retrieval-Augmented Generation	Kỹ thuật kết hợp tìm kiếm thông tin từ cơ sở dữ liệu với khả năng tạo sinh của LLM để cải thiện độ chính xác.
API	Application Programming Interface	Tập hợp quy tắc và giao thức cho phép các phần mềm giao tiếp với nhau.
REST	Representational State Transfer	Kiểu kiến trúc thiết kế API trên nền giao thức HTTP, dùng cho giao tiếp giữa giao diện web và máy chủ.

Từ viết tắt	Từ đầy đủ	Giải thích
SSR	Server-Side Rendering	Kỹ thuật kết xuất trang web phía máy chủ trước khi gửi về trình duyệt, giúp tối ưu tốc độ hiển thị lần đầu và SEO.
SEO	Search Engine Optimization	Tối ưu hóa để website thân thiện và dễ được các công cụ tìm kiếm xếp hạng cao.
SSE	Server-Sent Events	Cơ chế cho phép máy chủ đẩy dữ liệu liên tục về trình duyệt theo một chiều; được dùng cho luồng trả lời thời gian thực của trợ lý AI.
JWT	JSON Web Token	Chuẩn mã thông báo được ký số dùng để xác thực và truyền thông tin phiên đăng nhập một cách an toàn.
OAuth	Open Authorization	Giao thức ủy quyền cho phép đăng nhập thông qua tài khoản của bên thứ ba (ví dụ Google) mà không chia sẻ mật khẩu.
UI	User Interface	Giao diện người dùng – phần người dùng nhìn thấy và tương tác trực tiếp với hệ thống.
WebSocket	WebSocket	Giao thức truyền dữ liệu hai chiều theo thời gian thực giữa trình duyệt và máy chủ trên một kết nối bền; dùng cho thông báo tức thời.

Từ viết tắt	Từ đầy đủ	Giải thích
STOMP	Simple Text Oriented Messaging Protocol	Giao thức nhắn tin dạng văn bản chạy bên trên WebSocket, dùng để đẩy thông báo thời gian thực tới ứng dụng web.
SockJS	SockJS	Thư viện cung cấp kết nối kiểu WebSocket kèm cơ chế dự phòng (hỏi vòng) khi trình duyệt hoặc mạng không hỗ trợ WebSocket.
ICU	International Components for Unicode	Bộ định dạng chuỗi đa ngôn ngữ (số nhiều, tham số, ngày giờ) giúp hiển thị đúng ngữ pháp khi dịch giữa các ngôn ngữ.

Tóm tắt

Chương 1

Giới thiệu đề tài

1.1 Giới thiệu

1.2 Mục tiêu đề tài

1.3 Phạm vi đề tài

1.4 Phương pháp nghiên cứu

1.5 Ý nghĩa khoa học và thực tiễn

1.6 Cấu trúc báo cáo

Chương 2

Tổng quan

2.1 Tổng quan về hệ thống thuê nhà trọ trực tuyến

2.1.1 Khái niệm nhà trọ và thị trường cho thuê nhà trọ

2.1.2 Các mô hình hệ thống thuê nhà hiện nay

2.2 Khảo sát các nền tảng hiện có

Trước khi đề xuất giải pháp, nhóm tiến hành khảo sát một số nền tảng tiêu biểu trong lĩnh vực rao vặt và tìm kiếm bất động sản cho thuê, bao gồm ba nền tảng phổ biến tại Việt Nam (Batdongsan.com.vn, Phongtro123, Nhà Tốt) và một nền tảng quốc tế đại diện cho hướng ứng dụng dữ liệu – công nghệ ở mức cao (Zillow). Việc khảo sát nhằm nhận diện các tính năng đã được thị trường chấp nhận, đồng thời chỉ ra những hạn chế còn tồn tại để làm cơ sở cho định hướng của đề tài.

2.2.1 Batdongsan.com.vn

Batdongsan.com.vn [1] là một trong những nền tảng rao vặt bất động sản lớn và lâu đời nhất tại Việt Nam, cung cấp tin đăng cho cả nhu cầu mua bán lẫn cho thuê trên phạm vi toàn quốc. Điểm mạnh của nền tảng là kho tin đăng lớn, công cụ tìm kiếm và lọc theo nhiều tiêu chí (khu vực, khoảng giá, diện tích, loại hình), cùng mô hình kinh doanh dựa trên các gói tin VIP và đẩy tin giúp người đăng tăng khả năng tiếp cận. Tuy nhiên, nền tảng thiên về phục vụ môi giới chuyên nghiệp và phân khúc mua bán; trải nghiệm cho người thuê ở phân khúc nhà trọ, phòng trọ giá rẻ chưa phải là trọng tâm, và chất lượng tin đăng phụ thuộc nhiều vào người đăng.

2.2.2 Phongtro123

Phongtro123 [2] là nền tảng tập trung chuyên biệt vào phân khúc *phòng trọ, nhà trọ* và các loại hình cho thuê giá rẻ, hướng tới đối tượng sinh viên và người lao động. Giao diện đơn giản, dễ sử dụng, cho phép lọc nhanh theo khu vực và mức giá, phù hợp với nhu cầu tìm trọ phổ thông. Bù lại, nền tảng chủ yếu dừng ở vai trò bảng tin rao vặt: thông tin tin đăng còn sơ sài, thiếu các công cụ hỗ trợ ra quyết định như so sánh, định giá tham khảo, bản đồ trực quan hay gợi ý cá nhân hóa, và việc kiểm soát chất lượng/độ tin cậy của tin đăng còn hạn chế.

2.2.3 Nhà Tốt

Nhà Tốt [3] (chuyên mục bất động sản của nền tảng rao vặt Chợ Tốt) cung cấp tin đăng cho thuê và mua bán theo mô hình rao vặt giữa người dùng với người dùng. Nhờ thừa hưởng lượng người dùng lớn của Chợ Tốt, nền tảng có lưu lượng truy cập cao và đa dạng loại hình. Nền tảng có hỗ trợ bản đồ và một số tiện ích tìm kiếm, song giống các nền tảng nội địa khác, mức độ ứng dụng trí tuệ nhân tạo để định giá, gợi ý hay hỗ trợ hội thoại vẫn còn mờ nhạt, và trải nghiệm còn nặng tính rao vặt thủ công.

2.2.4 Zillow

Zillow [4] là nền tảng bất động sản hàng đầu tại Hoa Kỳ, nổi bật với việc ứng dụng dữ liệu và công nghệ ở mức cao. Tính năng tiêu biểu là *Zestimate* – mô hình ước tính giá trị bất động sản tự động dựa trên dữ liệu thị trường, cùng với bản đồ giàu thông tin, dữ liệu khu vực chi tiết và trải nghiệm người dùng được trau chuốt. Zillow cho thấy tiềm năng to lớn của việc tích hợp dữ liệu và AI vào lĩnh vực bất động sản. Tuy vậy, nền tảng được thiết kế cho thị trường Hoa Kỳ với đặc thù dữ liệu, pháp lý và hành vi người dùng khác biệt, do đó không trực tiếp phù hợp với thị trường thuê trọ tại Việt Nam.

2.2.5 So sánh ưu nhược điểm

Bảng 2.1 tổng hợp một số tiêu chí so sánh giữa các nền tảng đã khảo sát.

Bảng 2.1: So sánh các nền tảng tìm kiếm bất động sản cho thuê hiện có

Tiêu chí	Batdong-san	Phong-tro123	Nhà Tốt	Zillow
Tập trung nhà trọ giá rẻ	Trung bình	Cao	Trung bình	Thấp
Tìm kiếm & lọc đa tiêu chí	Cao	Trung bình	Trung bình	Cao
Bản đồ trực quan	Có	Hạn chế	Có	Cao
Định giá bằng AI	Không	Không	Không	Có
Gợi ý cá nhân hóa	Hạn chế	Không	Hạn chế	Có
Trợ lý hội thoại AI	Không	Không	Không	Hạn chế
Phù hợp thị trường Việt Nam	Cao	Cao	Cao	Thấp

Qua so sánh có thể thấy: các nền tảng nội địa (Batdongsan.com.vn,

Phongtro123, Nhà Tốt) phù hợp với thị trường và thói quen người dùng Việt Nam nhưng hầu như chưa khai thác trí tuệ nhân tạo để nâng cao trải nghiệm tìm kiếm và ra quyết định; trong khi đó, nền tảng quốc tế Zillow thể hiện rõ giá trị của việc tích hợp dữ liệu và AI nhưng lại không được thiết kế cho bối cảnh Việt Nam. Đây chính là khoảng trống mà đề tài hướng tới: kết hợp trải nghiệm phù hợp với người thuê trọ tại Việt Nam với các năng lực AI hiện đại.

2.3 Cơ sở lý thuyết về trí tuệ nhân tạo

2.4 Khoảng trống và cơ hội ứng dụng AI trong lĩnh vực cho thuê nhà trọ

2.5 Kết luận chương

Chương 3

Phân tích và đặc tả yêu cầu hệ thống

3.1 Mô tả bài toán

3.2 Phân tích nghiệp vụ

3.2.1 Người thuê trọ

3.2.2 Chủ nhà trọ

3.2.3 Quản trị viên

3.3 Yêu cầu chức năng

3.3.1 Đăng ký, đăng nhập

3.3.2 Quản lý hồ sơ người dùng

3.3.3 Đăng tin cho thuê phòng

3.3.4 Tìm kiếm phòng trọ

3.3.5 Đặt lịch xem phòng

3.3.6 Quản lý hợp đồng thuê

3.3.7 Đánh giá và phản hồi

Chương 4

Thiết kế hệ thống

4.1 Kiến trúc tổng thể hệ thống

4.1.1 Mô hình kiến trúc hệ thống

4.1.2 Khái quát các thành phần bên trong hệ thống

4.2 Thiết kế nền tảng Web

Nền tảng Web của hệ thống được tách thành hai ứng dụng độc lập nhưng dùng chung phong cách thiết kế và quy ước kỹ thuật: **ứng dụng web cho người dùng cuối** (webapp – nơi người thuê tìm kiếm, so sánh, lưu tin và liên hệ chủ trọ; đồng thời là không gian để người môi giới/chủ nhà đăng và quản lý tin) và **trang quản trị** (admin – dành cho quản trị viên). Phần này tập trung mô tả ứng dụng web cho người dùng cuối, vốn là thành phần người dùng tương tác trực tiếp nhiều nhất; phần trang quản trị được trình bày ở mục tiếp theo.

Webapp được xây dựng trên **Next.js 15** (theo mô hình Pages Router) cùng **React 19** và **TypeScript**, sử dụng Tailwind CSS v4 kết hợp bộ component shadcn/ui làm hệ thống giao diện. Việc lựa chọn Next.js cho phép kết xuất phía máy chủ (Server-Side Rendering – SSR) đối với các trang quan trọng về SEO như trang chi tiết tin đăng và trang tin tức,

đồng thời tận dụng được hệ sinh thái React phong phú cho phần tương tác phía trình duyệt [5], [6].

4.2.1 Tổ chức mã nguồn

Mã nguồn được tổ chức theo nguyên tắc *tách bạch mối quan tâm* (separation of concerns) và *thiết kế nguyên tử* (Atomic Design), với mục tiêu để mỗi tính năng có thể phát triển độc lập, dễ kiểm thử và dễ bảo trì. Toàn bộ mã nguồn nằm dưới thư mục `src/`, được phân chia thành các nhóm chính như Bảng 4.1.

Bảng 4.1: Cấu trúc thư mục mã nguồn của ứng dụng web

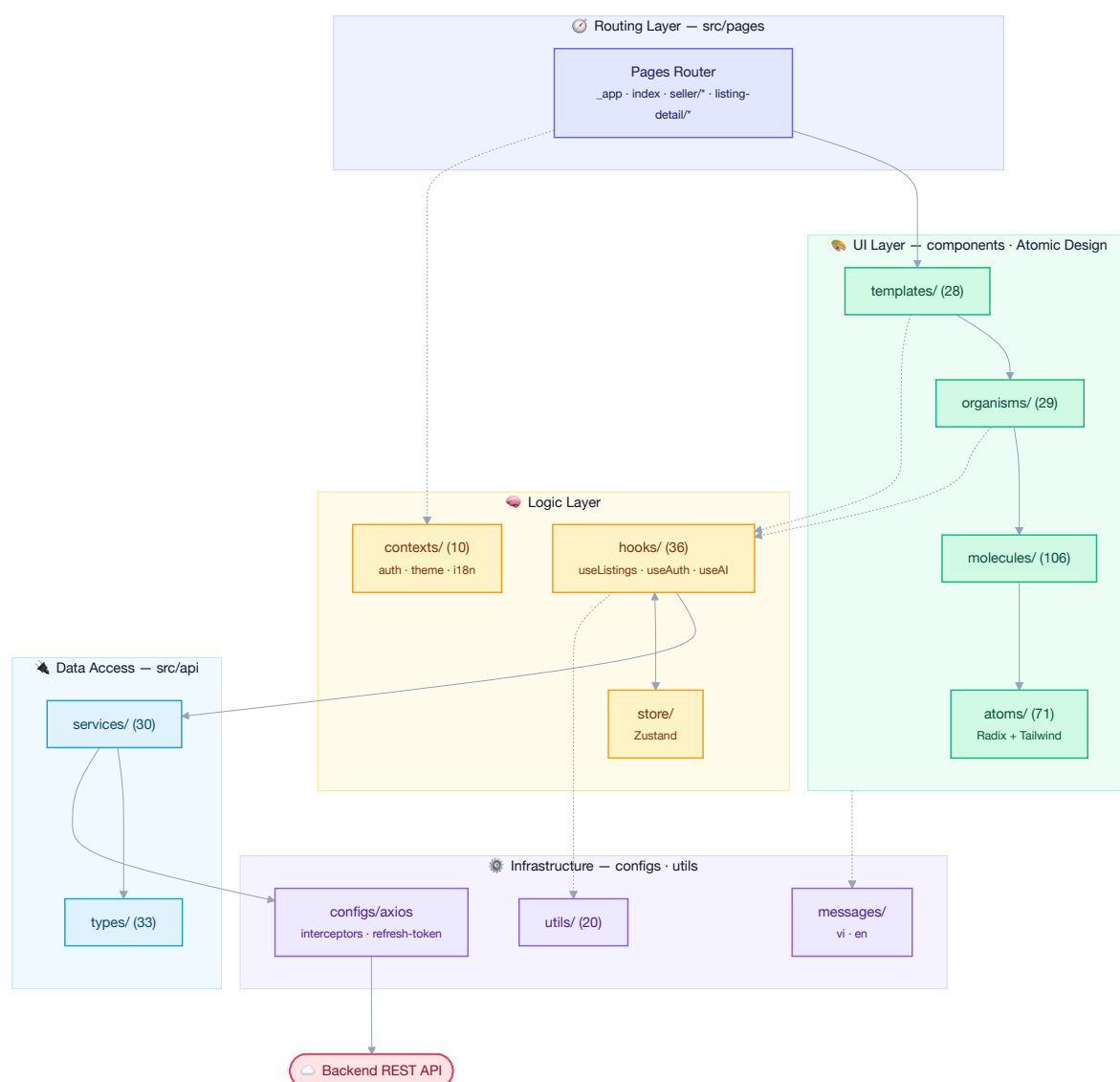
Thư mục	Vai trò
<code>pages/</code>	Lớp định tuyến (Pages Router). Mỗi tệp là một “vỏ mỏng” chỉ lắp ghép dữ liệu vào template và gắn bố cục trang.
<code>components/</code>	Cây giao diện theo Atomic Design: atoms (nguyên tử shadcn), molecules , organisms , templates và layouts .
<code>api/services/</code>	Các module gọi API theo từng nghiệp vụ (tin đăng, môi giới, thanh toán, tin tức, thông báo, AI...).
<code>hooks/</code>	Các React Hook tùy biến đóng gói logic nghiệp vụ và truy vấn dữ liệu.
<code>contexts/, store/</code>	Quản lý trạng thái dùng chung (xác thực, ngôn ngữ, danh sách so sánh...) qua React Context và Zustand.
<code>configs/, constants/</code>	Cấu hình thư viện (axios, biến môi trường) và hằng số toàn cục.
<code>messages/, utils/, styles/</code>	Bộ từ điển đa ngôn ngữ, các hàm tiện ích thuần và mã định kiểu giao diện.

Cây giao diện tuân thủ nghiêm ngặt thứ tự phân lớp **atoms** → **molecules** → **organisms** → **templates** → **pages**, trong đó lớp dưới không bao giờ được phép nhập (import) từ lớp trên. Cụ thể, **atoms** là các nguyên tử

giao diện cơ bản (nút bấm, ô nhập liệu, hộp thoại...) được chuẩn hóa theo shadcn/ui; **molecules** ghép các nguyên tử thành các khối nhỏ như trường biểu mẫu, thẻ lọc; **organisms** là các khối tính năng hiểu biết về dữ liệu và quy tắc nghiệp vụ (thẻ tin đăng, hộp thoại đăng nhập, widget trợ lý AI...); **templates** là bố cục đầy đủ của một họ trang; còn **pages** chỉ đóng vai trò lắp ráp. Mỗi thư mục thành phần đều cung cấp một tệp **index** làm điểm xuất khẩu tập trung (barrel export), giúp việc nhập khẩu nhất quán và che giấu cấu trúc nội bộ.

Song song với cây giao diện, logic được tách theo *ngành vụ*: mỗi miền dữ liệu có một module dịch vụ (***.service.ts**) chịu trách nhiệm gọi API và một nhóm hook (**use***) bao bọc dịch vụ đó. Nhờ vậy, logic nghiệp vụ nằm ở hook và context, việc lấy dữ liệu nằm ở tầng dịch vụ, còn component chỉ lo phần hiển thị. Cách phân tách này giúp giảm thiểu sự phụ thuộc chéo và cho phép nhiều thành viên cùng phát triển các tính năng khác nhau mà ít xảy ra xung đột.

Hình 4.1 tổng hợp cách tổ chức mã nguồn webapp theo các lớp nêu trên. Lớp định tuyến (**pages**) nằm trên cùng; bên dưới là cây giao diện Atomic Design (**templates** → **organisms** → **molecules** → **atoms**) tách biệt với nhánh logic gồm **hooks**, **contexts/store** và tầng dịch vụ (**api/services**) chịu trách nhiệm gọi API qua **axios**. Mũi tên nét liền thể hiện quan hệ phụ thuộc khi kết xuất giao diện, còn mũi tên nét đứt thể hiện việc các thành phần sử dụng hook để lấy dữ liệu; nhờ đó component chỉ lo phần hiển thị, còn logic nghiệp vụ và truy xuất dữ liệu được đẩy xuống các tầng phía dưới.



Hình 4.1: Sơ đồ tổ chức mã nguồn ứng dụng web (webapp)

4.2.2 Cơ chế kết nối và tích hợp

Ứng dụng web không tự chứa dữ liệu mà kết nối tới nhiều dịch vụ phía sau thông qua các kênh giao tiếp được chuẩn hóa:

- **Giao tiếp HTTP với Backend chính.** Mọi yêu cầu REST đi qua một thẻ hiện `axios` duy nhất được cấu hình tập trung tại `src/configs/axios`. Thẻ hiện này gắn sẵn URL gốc của Backend

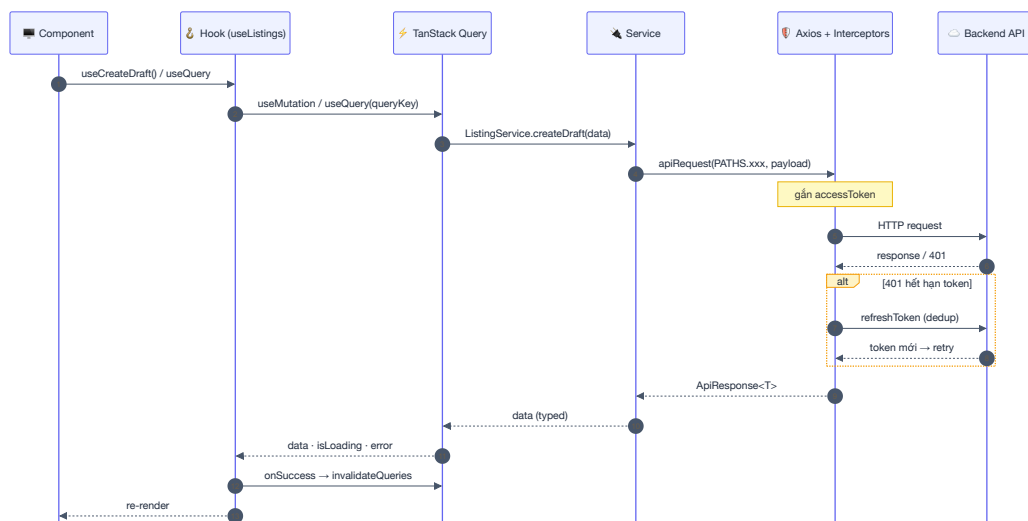
(biến môi trường `URL_API_BASE`), tiêu đề mặc định và bộ chặn (interceptor) xử lý xác thực. Toàn bộ phản hồi được chuẩn hóa về một kiểu `ApiResponse` thống nhất để các tầng phía trên xử lý nhất quán.

- **Xác thực bằng JWT và cơ chế làm mới token.** Bộ chặn yêu cầu tự động đính kèm *access token* vào tiêu đề **Authorization**. Khi token sắp hết hạn hoặc máy chủ trả về lỗi 401, hệ thống tự động gọi *refresh token* để lấy token mới rồi phát lại yêu cầu ban đầu một cách trong suốt với người dùng. Cơ chế này còn khử trùng lặp (deduplication) để nhiều yêu cầu đồng thời chỉ kích hoạt đúng một lần làm mới, tránh tình trạng đua tranh (race condition).
- **Cổng chặn truy cập (middleware).** Tập `src/middleware.ts` đóng vai trò cổng kiểm soát: nó cho phép một danh sách các tuyến công khai (trang chủ, danh sách tin, chi tiết tin, tin tức, bản đồ...) và chặn các tuyến cá nhân khi người dùng chưa đăng nhập. Khi đó, người dùng được chuyển hướng kèm tham số `returnUrl` để sau khi đăng nhập có thể quay lại đúng trang đang truy cập.
- **Bộ nhớ đệm dữ liệu máy chủ.** Toàn bộ trạng thái lấy từ máy chủ được quản lý bởi **TanStack Query**, giúp tự động lưu đệm, làm mới nền và đồng bộ dữ liệu giữa các thành phần. Một số dữ liệu thống kê ổn định theo ngày (ví dụ thống kê tin theo tỉnh thành) còn được lưu bền vào `localStorage` để tồn tại qua các lần tải lại trang.
- **Kết nối dịch vụ AI bằng luồng SSE.** Trợ lý hội thoại AI kết nối tới một dịch vụ AI riêng (biến môi trường `URL_API_AI`) và nhận câu trả lời theo thời gian thực qua kỹ thuật *Server-Sent Events* (sử dụng thư viện `fetch-event-source`). Nhờ đó nội dung trả lời hiện dần theo từng đoạn, kèm theo trạng thái công cụ mà AI đang thực thi, và người dùng có thể hủy luồng đang chạy.
- **Thông báo thời gian thực qua WebSocket.** Các thông báo (tin

được duyệt, có người quan tâm...) được đẩy theo thời gian thực qua giao thức **STOMP** trên **SockJS**, kết hợp với cơ chế hỏi vòng (polling) dự phòng khi mất kết nối.

- **Tích hợp dịch vụ bên thứ ba.** Bản đồ tin đăng tích hợp Google Maps; luồng thanh toán điều hướng sang cổng thanh toán rồi nhận lại kết quả qua một trang callback công khai để cổng có thể chuyển hướng về.

Hình 4.2 minh họa luồng dữ liệu điển hình khi thực hiện một thao tác đọc/ghi: thành phần giao diện gọi hook, hook ủy thác cho TanStack Query, qua tầng dịch vụ và **axios** (kèm bộ chặn tự đính kèm và làm mới token) trước khi tới Backend. Khi ghi dữ liệu thành công, hook chủ động vô hiệu hóa bộ đệm liên quan (**invalidateQueries**) để dữ liệu hiển thị được làm mới một cách nhất quán.



Hình 4.2: Luồng dữ liệu giữa giao diện, hook, tầng dịch vụ và Backend

4.2.3 Hỗ trợ đa ngôn ngữ

Ứng dụng hỗ trợ song ngữ **tiếng Việt** (mặc định) và **tiếng Anh**, được triển khai bằng thư viện **next-intl**. Mọi chuỗi hiển thị cho người dùng –

nhân, nút bấm, thông báo, thông điệp lỗi, văn bản trạng thái rỗng – đều không được viết cứng trong mã mà phải lấy qua các hàm `useTranslations` (phía client) hoặc `getTranslations` (phía server), tham chiếu tới hai bộ từ điển `messages/vi.json` và `messages/en.json`.

Một số điểm kỹ thuật đáng chú ý trong cơ chế đa ngôn ngữ:

- **Tải từ điển theo nhu cầu.** Vì phần lớn người dùng sử dụng tiếng Việt, bộ từ điển `vi.json` được đóng gói sẵn cùng ứng dụng; bộ `en.json` chỉ được tải động khi người dùng thực sự chuyển sang tiếng Anh, giúp giảm kích thước gói tải ban đầu.
- **Bảo đảm đồng bộ khóa dịch.** Một kịch bản kiểm tra (`check-i18n-keys.js`) được chạy tự động trong bước `pre-commit` để đối chiếu sự tương đồng khóa giữa hai tệp ngôn ngữ; nếu thiếu khóa ở một ngôn ngữ, thao tác commit sẽ bị chặn.
- **Dùng định dạng ICU.** Các chuỗi có tham số sử dụng placeholder kiểu ICU (`{name}`, `{count}`, `plural`, ...) thay vì nối chuỗi thủ công, bảo đảm đúng ngữ pháp khi trật tự từ khác nhau giữa các ngôn ngữ.
- **Quản lý trạng thái ngôn ngữ.** Việc chuyển ngôn ngữ được điều phối qua một context riêng (`SwitchLanguageProvider`), ghi nhớ lựa chọn của người dùng và cung cấp bộ từ điển phù hợp cho toàn bộ cây thành phần.

4.2.4 Ưu điểm

Cách tổ chức và tích hợp nêu trên mang lại một số ưu điểm rõ rệt cho nền tảng web:

- **Dễ bảo trì và mở rộng.** Kiến trúc Atomic Design phân lớp rõ ràng cùng việc tách logic theo nghiệp vụ giúp định vị và sửa đổi mã

nhANH chóng; thêm một tính năng mới thường chỉ cần bổ sung một template, một dịch vụ và một nhóm hook tương ứng mà không ảnh hưởng phần còn lại.

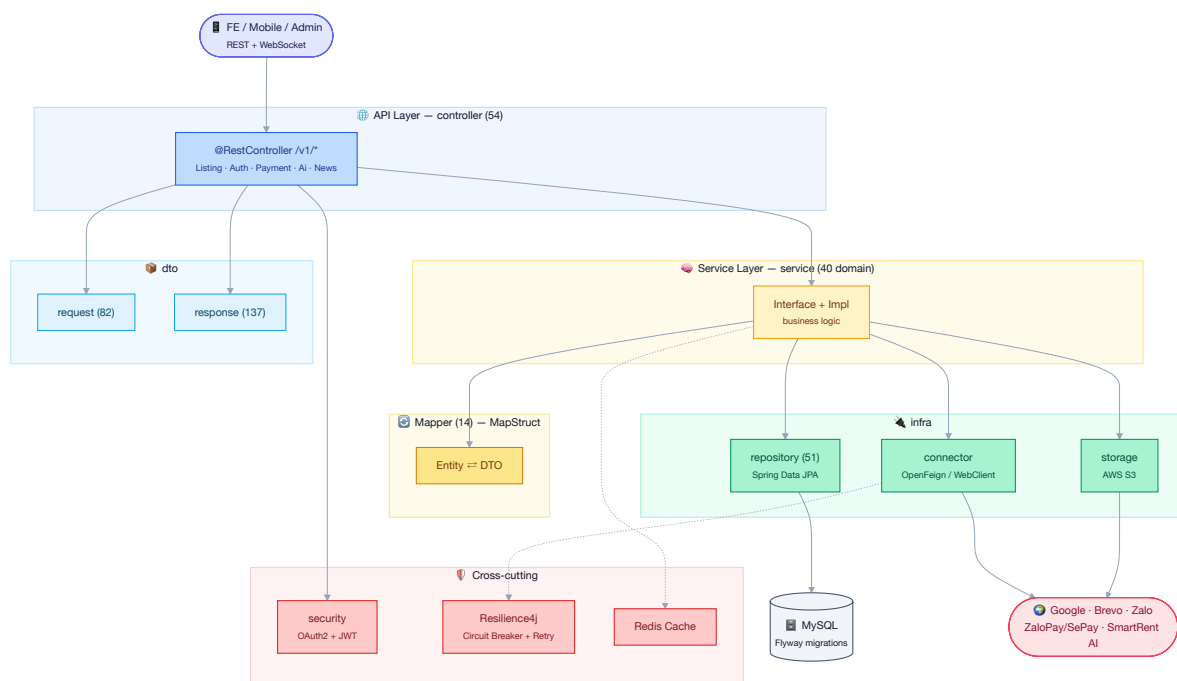
- **An toàn kiểu dữ liệu.** Việc dùng TypeScript trên toàn dự án, giới hạn kiểu `any` chỉ trong tầng định nghĩa dữ liệu truyền tải, giúp phát hiện sớm lỗi ngay khi biên dịch.
- **Hiệu năng và trải nghiệm tốt.** Kết hợp SSR cho các trang trọng yếu về SEO, tải động (lazy loading) các thành phần nặng như bản đồ và biểu đồ, cùng cơ chế lưu đệm của TanStack Query giúp trang hiển thị nhanh và mượt.
- **Nhất quán về giao diện.** Hệ thống thiết kế tập trung (typography, khoảng cách, màu sắc theo token ngữ nghĩa) cùng các quy tắc lint riêng bảo đảm giao diện đồng nhất giữa các trang và giữa các thành viên phát triển.
- **Thuận lợi cho cộng tác nhóm.** Quy ước thư mục, barrel export và các cổng kiểm soát chất lượng (lint, kiểm tra kiểu, kiểm tra khóa đa ngôn ngữ) tạo nền tảng để nhiều người cùng làm việc trên cùng mã nguồn mà vẫn giữ được chất lượng ổn định.

4.3 Thiết kế hệ thống Backend

Backend của hệ thống được xây dựng trên **Spring Boot 3.4** (Java) và đóng gói thành một khối triển khai duy nhất theo mô hình **Modular Monolith**: toàn bộ nghiệp vụ chạy trong cùng một tiến trình nhưng được chia thành nhiều module theo miền nghiệp vụ, với ranh giới được giữ rõ ràng ở mức gói (package) và tầng dịch vụ. Cách tiếp cận này vẫn giữ được sự đơn giản khi vận hành và gỡ lỗi của một monolith, đồng thời đạt mức tách bạch đủ tốt để nhiều thành viên cùng phát triển song song.

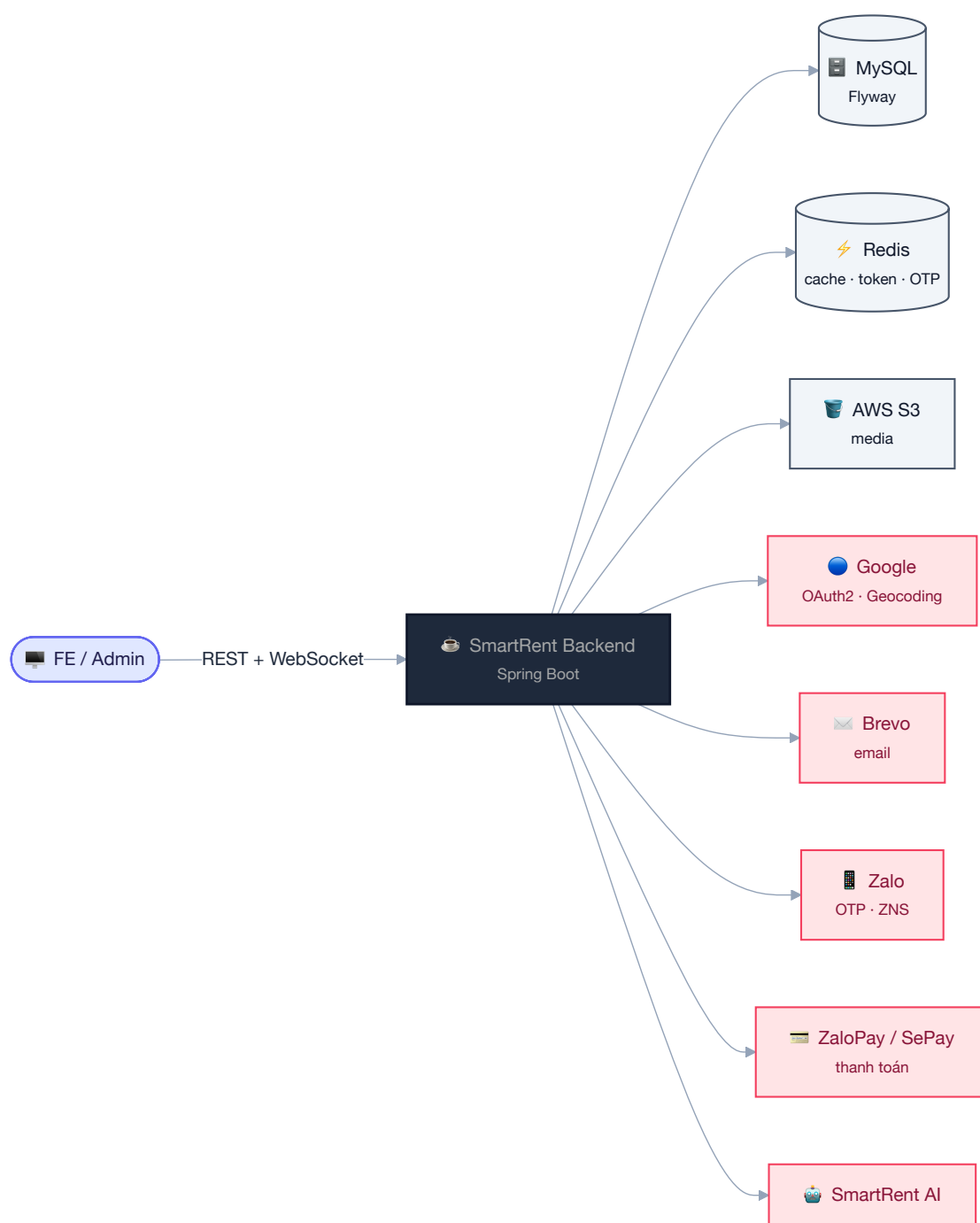
4.3.1 Kiến trúc Modular Monolith

Hình 4.3 mô tả kiến trúc phân lớp của Backend. Mọi yêu cầu từ web, di động hay trang quản trị đi vào qua tầng API gồm 54 controller (@RestController, đa số định tuyến theo tiền tố /v1); sau khi qua bộ lọc bảo mật, yêu cầu được chuyển xuống tầng dịch vụ (chứa toàn bộ logic nghiệp vụ), rồi tới tầng truy xuất dữ liệu (repository) và cơ sở dữ liệu MySQL. Bao quanh các tầng là những mối quan tâm xuyên suốt (cross-cutting): xác thực OAuth2 kết hợp JWT, bộ đệm Redis và cơ chế chịu lỗi Resilience4j.



Hình 4.3: Kiến trúc phân lớp của hệ thống Backend (Modular Monolith)

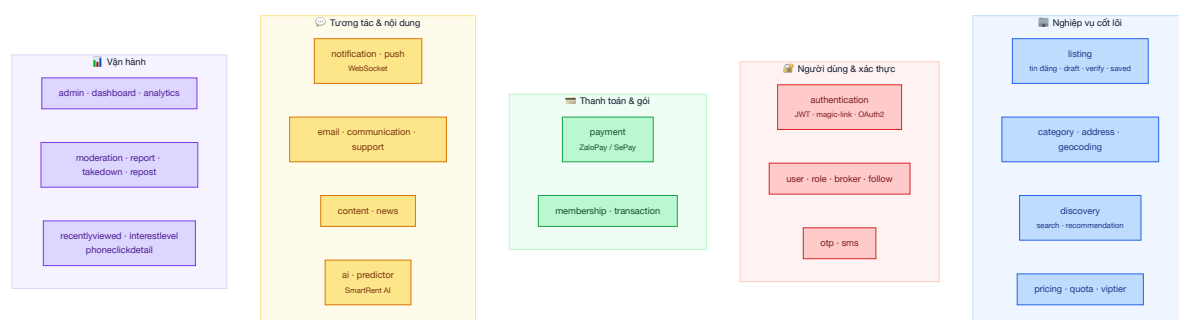
Tuy là một monolith nhưng hệ thống không cô lập: nó kết nối tới nhiều dịch vụ bên ngoài thông qua tầng **connector** (OpenFeign/WebClient) và **storage**, như tóm tắt ở Hình 4.4 – gồm MySQL, Redis, lưu trữ đối tượng (S3), đăng nhập và định vị qua Google, gửi email (Brevo), gửi OTP qua Zalo, cổng thanh toán (ZaloPay/SePay) và dịch vụ AI riêng.



Hình 4.4: Các dịch vụ bên ngoài mà Backend tích hợp

4.3.2 Các module nghiệp vụ

Phần nghiệp vụ được chia thành khoảng 40 module dịch vụ, mỗi module phụ trách một miền độc lập. Hình 4.5 nhóm các module theo năm cụm chức năng chính: (i) nghiệp vụ cốt lõi về tin đăng, danh mục, tìm kiếm và định giá; (ii) người dùng và xác thực; (iii) thanh toán và gói thành viên; (iv) tương tác và nội dung (thông báo, email, tin tức, AI); và (v) vận hành (quản trị, kiểm duyệt, thống kê hành vi).



Hình 4.5: Các module nghiệp vụ của Backend gom theo cụm chức năng

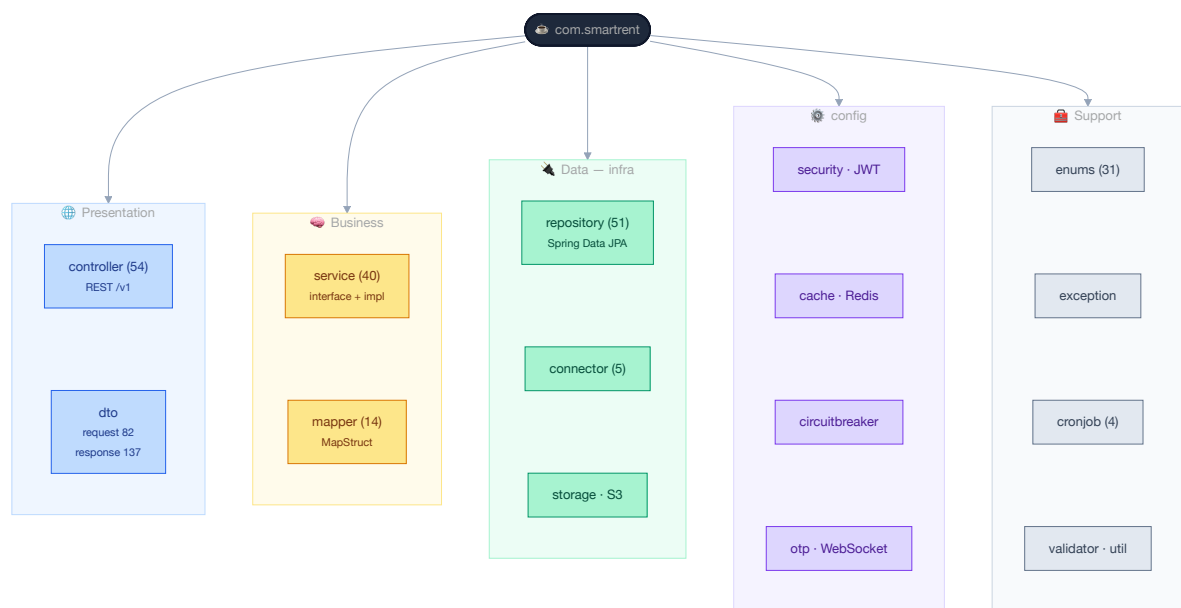
4.3.3 Tổ chức phân tầng bên trong module

Bên trong mỗi module, mã nguồn tuân theo cùng một khuôn phân tầng nhất quán, phản ánh qua cấu trúc gói `com.smartrent` ở Hình 4.6. Tầng **controller** chỉ tiếp nhận yêu cầu và kiểm tra hợp lệ dữ liệu (`@Valid`); tầng **service** (tách giao diện và phần hiện thực `impl`) chứa logic nghiệp vụ; **mapper** (MapStruct) chuyển đổi giữa thực thể và DTO; còn **infra/repository** dùng Spring Data JPA để truy xuất dữ liệu. Các đối tượng truyền tải được tách thành `dto/request` và `dto/response`, trong khi cấu hình hệ thống (bảo mật, cache, circuit breaker, OTP, WebSocket) nằm trong gói `config`. Bảng 4.2 liệt kê chi tiết vai trò của từng gói chính trong mã nguồn Backend.

Bảng 4.2: Cấu trúc gói mã nguồn của Backend (`com.smartrent`)

Gói	Vai trò
<code>controller/</code>	Lớp API: 54 lớp <code>@RestController</code> expose các REST endpoint (đa số theo tiền tố <code>/v1</code>); tiếp nhận yêu cầu, kiểm tra hợp lệ dữ liệu và ủy thác cho tầng dịch vụ.
<code>service/</code>	Lớp nghiệp vụ: 40 module theo miền (tin đăng, xác thực, thanh toán, AI...), mỗi module tách <i>giao diện</i> và phần <i>hiện thực</i> (<code>impl</code>).
<code>dto/</code>	Đối tượng truyền tải, tách <code>request</code> (82) và <code>response</code> (137); cách ly mô hình API khỏi thực thể cơ sở dữ liệu.
<code>mapper/</code>	14 bộ chuyển đổi (<code>MapStruct</code>) giữa thực thể (<code>entity</code>) và DTO.
<code>infra/</code>	Hạ tầng truy xuất dữ liệu và tích hợp: <code>repository</code> (51 – Spring Data JPA), <code>connector</code> (gọi dịch vụ ngoài qua <code>OpenFeign/WebClient</code>), <code>storage</code> (S3), <code>client</code> .
<code>config/</code>	Cấu hình hệ thống: <code>security</code> (OAuth2 + JWT), <code>cache</code> (Redis), <code>circuitbreaker</code> (Resilience4j), <code>otp</code> , <code>WebSocket</code> , S3 và cổng thanh toán.
<code>enums/</code> , <code>constants/</code>	31 kiểu liệt kê và các hằng số nghiệp vụ dùng chung.
<code>exception/</code>	Các ngoại lệ nghiệp vụ và bộ xử lý lỗi tập trung (<code>GlobalPaymentExceptionHandler</code>).
<code>cronjob/</code>	Tác vụ định kỳ: tự kiểm duyệt tin bằng AI, thông báo tin sắp hết hạn, làm mới bộ đệm thống kê trang chủ.
<code>util/</code> , <code>utility/</code> , <code>validator/</code>	Hàm tiện ích (sinh <code>Snowflake</code> ID, chuẩn hóa văn bản tìm kiếm, dựng email, che dữ liệu nhạy cảm) và bộ kiểm tra ràng buộc (số tiền, hạn mức).

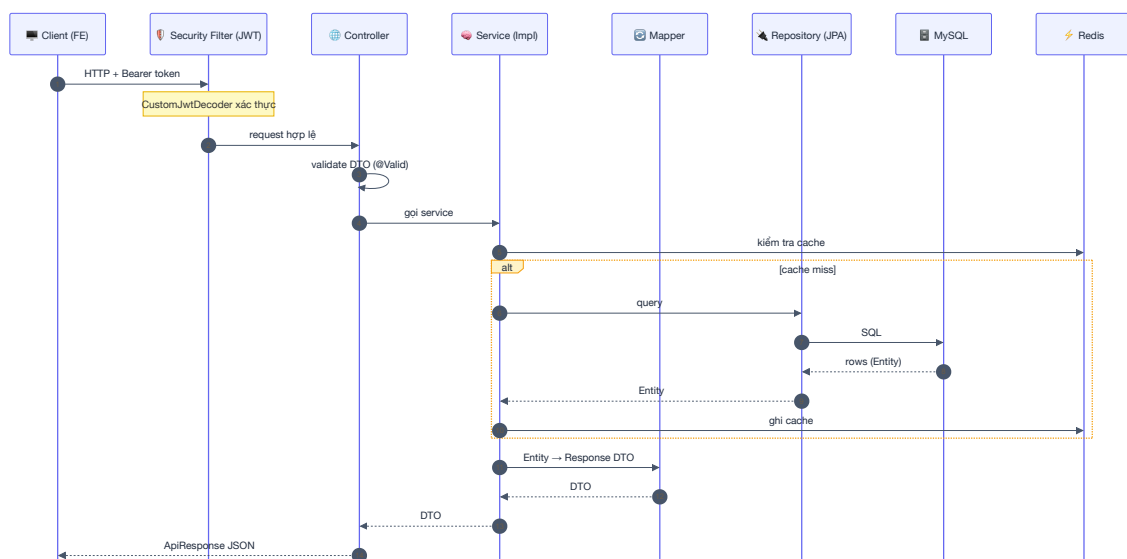
Cấu trúc gói nói trên được minh họa trực quan ở Hình 4.6.



Hình 4.6: Cấu trúc gói `com.smartrent` và cách phân tầng

4.3.4 Luồng dữ liệu

Hình 4.7 minh họa luồng xử lý một yêu cầu có xác thực. Yêu cầu kèm *Bearer token* trước hết đi qua bộ lọc bảo mật để `CustomJwtDecoder` kiểm tra token; controller kiểm tra hợp lệ DTO rồi gọi service. Service ưu tiên đọc từ bộ đệm Redis, chỉ truy vấn cơ sở dữ liệu khi *cache miss* và ghi lại kết quả vào đệm; sau đó dùng mapper chuyển thực thể sang DTO phản hồi và trả về cho client dưới dạng `ApiResponse` JSON.



Hình 4.7: Luồng xử lý một yêu cầu có xác thực ở Backend

4.3.5 Ưu điểm

Mô hình Modular Monolith mang lại một số ưu điểm cho Backend của hệ thống:

- **Đơn giản khi vận hành.** Một khối triển khai duy nhất giúp việc build, kiểm thử và gỡ lỗi dễ dàng, đồng thời giảm chi phí hạ tầng so với kiến trúc microservices.
- **Tách bạch theo miền.** Ranh giới module rõ ràng cho phép nhiều thành viên cùng phát triển song song mà ít giẫm chân lên nhau.
- **Nhất quán phân tầng.** Cùng một khuôn `controller-service-repository` ở mọi module giúp mã dễ đọc, dễ bảo trì và rút ngắn thời gian làm quen cho thành viên mới.
- **Chịu lỗi và hiệu năng.** Bộ đệm Redis giảm tải truy vấn cơ sở dữ liệu, còn Resilience4j (circuit breaker và retry) cô lập sự cố từ các dịch vụ bên ngoài, giữ hệ thống ổn định.

- **Bảo mật tập trung.** Cơ chế xác thực OAuth2 kết hợp JWT được áp dụng thống nhất ở tầng lọc, tách khỏi logic nghiệp vụ.

4.4 Thiết kế kiến trúc AI

4.4.1 Luồng xử lý AI Chatbot

4.4.2 Luồng gợi ý phòng trọ

4.4.3 Tích hợp API AI

4.4.4 Kiến trúc tích hợp AI

4.5 Thiết kế cơ sở dữ liệu

4.5.1 Giới thiệu tổng quan

4.5.2 Sơ đồ thiết kế cơ sở dữ liệu của hệ thống chính

4.5.3 Mô tả chi tiết các bảng

4.6 Thiết kế giao diện

4.6.1 Giao diện người thuê

Giao diện dành cho người thuê được thiết kế theo định hướng *ưu tiên thiết bị di động* (mobile-first) và đáp ứng (responsive) trên mọi kích thước màn hình, lấy người tìm trọ làm trung tâm của hành trình trải nghiệm. Toàn bộ giao diện kế thừa một **hệ thống thiết kế** (design system) thống nhất: bảng màu, kiểu chữ, bo góc và khoảng cách đều được khai báo dưới dạng *token ngữ nghĩa* thông qua các biến CSS (CSS custom properties) trong `styles/globals.css`, thay vì gán giá trị màu cứng tại từng thành

phần. Nhờ đó, hệ thống hỗ trợ sẵn hai chế độ **sáng/tối** (light/dark) chỉ bằng cách hoán đổi nhóm biến, và giữ được sự đồng nhất về thị giác giữa các trang cũng như giữa các thành viên phát triển.

Về **kiểu chữ**, ứng dụng dùng bộ phông hỗ trợ tốt cả tiếng Việt lẫn tiếng Anh (Be Vietnam Pro làm phông chính, Inter và Plus Jakarta Sans cho tiêu đề), tải qua cơ chế tối ưu phông của Next.js để tránh hiện tượng nhấp nháy khi tải. Các thành phần giao diện được xây trên nền **shadcn/ui** kết hợp Tailwind CSS v4, bảo đảm khả năng truy cập (accessibility) và tính nhất quán của nút bấm, biểu mẫu, hộp thoại, thẻ thông tin...

Bố cục chung của khu vực người thuê được gói trong một *layout* dùng lại cho hầu hết các trang, gồm: **thanh điều hướng trên cùng** (header) chứa logo, thanh tìm kiếm, bộ chuyển ngôn ngữ, chuông thông báo và khu vực tài khoản; **phần nội dung** ở giữa; **chân trang** (footer) với các liên kết phụ trợ; và một **thanh nổi so sánh** hiển thị khi người dùng đang chọn tin để đối chiếu. Cách bố trí này giúp người dùng luôn truy cập nhanh các chức năng cốt lõi ở bất kỳ trang nào.

Một số nguyên tắc thiết kế giao diện được áp dụng xuyên suốt khu vực người thuê:

- **Phản hồi trạng thái rõ ràng.** Mỗi thao tác bất đồng bộ đều có trạng thái tương ứng: khung chờ dạng *skeleton* khi đang tải dữ liệu, chỉ báo tiến trình khi chuyển trang, thông báo lỗi thân thiện và *trạng thái rỗng* có hướng dẫn khi không có kết quả.
- **Tải động thành phần nặng.** Các khối tốn tài nguyên như bản đồ, biểu đồ lịch sử giá và danh sách tin tương tự được tải động (lazy loading) kèm khung chờ, giúp nội dung chính hiển thị nhanh mà không bị chặn.
- **Bố cục thích ứng.** Trên màn hình lớn, trang danh sách tin dùng bố cục hai cột (bộ lọc cạnh kết quả); trên thiết bị di động, bộ lọc thu gọn vào hộp thoại trượt để tiết kiệm không gian.

- **Đa ngôn ngữ và bản địa hóa.** Mọi nhãn, thông báo và định dạng (tiền tệ, ngày tháng) đều đi qua lớp đa ngôn ngữ Việt – Anh, cho phép chuyển đổi tức thời mà không cần tải lại trang.

Các màn hình chính của người thuê – trang chủ, danh sách/tìm kiếm tin, chi tiết tin đăng, bản đồ, so sánh, tin đã lưu, trang theo dõi người đăng và tin tức – đều tuân theo cùng bộ nguyên tắc trên; kết quả hiện thực cụ thể của từng màn hình được trình bày ở Chương 5 (phần kết quả giao diện dành cho người thuê).

4.6.2 Giao diện chủ trọ

4.6.3 Giao diện quản trị

4.6.4 Giao diện chatbot AI

Chương 5

Xây dựng và triển khai hệ thống

5.1 Môi trường phát triển

5.2 Các giai đoạn của đề tài

5.2.1 Xây dựng phiên bản MVP (Tháng 12/2025 – 02/2026)

5.2.2 Hoàn thiện các chức năng nghiệp vụ (Tháng 02/2025 – 04/2026)

5.2.3 Tích hợp AI nâng cao, thanh toán và hoàn thiện hệ thống (Tháng 05/2026 – 07/2026)

5.3 Quy trình làm việc nhóm

5.3.1 Phân công vai trò

5.3.2 Phương pháp làm việc nhóm

Để đảm bảo dự án vận hành trơn tru với 6 thành viên và khối lượng công nghệ phức tạp, nhóm đã thiết lập một quy trình làm việc chặt chẽ

dựa trên các nguyên tắc và công cụ sau:

a. Nguyên tắc cộng tác

Nhóm tuân thủ nguyên tắc “Độc lập trong thực thi – Thống nhất trong kết quả”. Mỗi thành viên được trao quyền chủ động hoàn toàn trong phạm vi Module mình phụ trách nhưng phải đảm bảo tuân thủ các chuẩn giao tiếp và quy ước mã nguồn đã thống nhất từ đầu. Mọi vấn đề phát sinh liên quan đến kiến trúc chung đều phải được đưa ra thảo luận tập thể trước khi ra quyết định thay đổi.

b. Các kênh giao tiếp và công cụ quản lý

- **Trao đổi thường xuyên:** Sử dụng Discord làm kênh giao tiếp chính thức. Các kênh được phân chia rõ ràng theo từng chủ đề: #biz (trao đổi nghiệp vụ, yêu cầu chức năng), #bug (báo lỗi) và #document (chia sẻ tài liệu, liên kết liên quan).
- **Nhắn tin nhanh:** Sử dụng Messenger và Zalo cho các trao đổi nhanh, thông báo gấp và liên lạc khi cần phản hồi tức thời.
- **Họp trực tuyến:** Sử dụng Discord cho các buổi họp định kỳ và các buổi họp đột xuất khi cần giải quyết vấn đề kỹ thuật phức tạp.
- **Quản lý tài liệu:** Sử dụng Google Drive để lưu trữ tập trung toàn bộ tài liệu khảo sát, đặc tả yêu cầu, thiết kế giao diện và biên bản cuộc họp.
- **Quản lý công việc:** Sử dụng Jira để theo dõi trạng thái công việc, đảm bảo trực quan hóa tiến độ dự án.

c. Quy trình quản lý mã nguồn

Nhóm sử dụng Git và nền tảng GitHub để quản lý mã nguồn với quy trình nghiêm ngặt:

- **Chiến lược nhánh phát triển:** Không commit mã nguồn trực tiếp lên nhánh chính (main). Mỗi tính năng mới hoặc bản sửa lỗi phải được phát triển trên một nhánh riêng biệt (feature/tên-tính-năng hoặc fix/tên-lỗi).
- **Kiểm soát chất lượng mã nguồn:** Để hợp nhất mã nguồn vào nhánh chính, thành viên phải tạo pull request. Yêu cầu này bắt buộc phải được đánh giá và chấp thuận bởi ít nhất 1 thành viên khác để đảm bảo không có lỗi logic và tuân thủ các quy tắc chung.

5.3.3 Phân chia nhiệm vụ và theo dõi tiến độ

a. Mô hình phát triển

Nhóm áp dụng quy trình Scrum trong việc phát triển đề tài, với độ dài mỗi sprint là hai tuần. Như đã đề cập ở phần trước, nhóm sử dụng Jira để lên kế hoạch, phân chia và quản lý công việc. Mỗi hai tuần một lần, nhóm tổ chức một buổi họp trực tiếp (hoặc trực tuyến khi cần thiết) nhằm thực hiện Sprint Review và Sprint Planning, bao gồm các hoạt động như sau:

Cuộc họp sẽ bắt đầu bằng việc kiểm tra tiến độ của sprint trước đó. Quá trình lên kế hoạch có thể bị ảnh hưởng bởi tiến độ của các công việc chưa hoàn thành. Vì vậy, cuối mỗi sprint, các thành viên sẽ báo cáo tiến độ công việc của mình cho toàn nhóm, bao gồm những công việc đã hoàn thành, chưa hoàn thành và những khó khăn gặp phải trong quá trình thực hiện. Ngoài ra, các thành viên khác sau khi nghe trình bày cũng có thể nắm được khái quát cách thực hiện, và cho nhận xét, góp ý nếu cần.

Tiếp đến, nhóm trưởng sẽ dựa theo roadmap, các công việc chưa hoàn thành, và các vấn đề mới phát sinh trong sprint vừa qua để quyết định mục tiêu chung cho sprint kế tiếp và lên danh sách công việc. Đối với các công việc chưa hoàn thành, chúng vẫn sẽ được liệt kê vào danh sách nói trên, nhằm đảm bảo sự minh bạch và tính chính xác. Điều này còn giúp tránh việc quên mất công việc hay quá tải cho các thành viên.

Cuối cùng, các công việc sẽ được phân công cho các thành viên giữ vai trò phù hợp và có sự đồng thuận chung của cả nhóm. Các công việc chưa hoàn thành ở sprint trước có thể cân nhắc cho thành viên khác đảm nhận, tùy vào tình huống thực tế.

Bên cạnh cuộc họp định kỳ mỗi sprint, nhóm còn thực hiện Daily Standup thông qua tin nhắn trên nhóm chat Discord. Mỗi ngày, mỗi thành viên sẽ cập nhật ngắn gọn về công việc đang thực hiện, trạng thái hiện tại của công việc, và các blocker (nếu có). Hình thức này giúp cả nhóm nắm bắt tình hình kịp thời, phát hiện sớm các vấn đề và hỗ trợ lẫn nhau một cách hiệu quả mà không cần tổ chức họp mỗi ngày.

b. Quy trình theo dõi và báo cáo

Tiến độ công việc được theo dõi sát sao thông qua bảng điều khiển công việc chung trên Jira. Mỗi thành viên cần cập nhật tình trạng công việc trên bảng này hàng ngày. Trưởng nhóm có trách nhiệm cập nhật trạng thái tổng thể và báo cáo định kỳ cho Giảng viên hướng dẫn.

Các trạng thái công việc được định nghĩa rõ ràng:

- **Backlog:** Công việc đã được mô tả sơ bộ nhưng chưa được đưa vào kế hoạch sprint vì các yếu tố như yêu cầu chưa rõ ràng, tính khả thi, chi phí, hoặc độ ưu tiên thấp.
- **To do:** Công việc đã có mô tả chi tiết, được ước lượng thời gian/công sức, và sẵn sàng để phân công cho thành viên thực hiện trong sprint hiện tại hoặc sprint tiếp theo.
- **In progress:** Công việc đang được thành viên trong nhóm tiến hành thực hiện.
- **Testing:** Công việc đã hoàn thành về mặt cơ bản (viết mã nguồn hoặc thiết kế), cần được kiểm thử, nhận phản hồi và góp ý từ các thành viên khác. Nhóm ưu tiên kiểm tra sớm nhất để kịp thời sửa lỗi (nếu có).

- **Done:** Công việc đã hoàn tất và đáp ứng đầy đủ các yêu cầu đề ra.

Việc phân chia nhiệm vụ và quy trình làm việc khoa học này là nền tảng quan trọng giúp nhóm kiểm soát được khối lượng công việc lớn, đảm bảo sự đồng bộ giữa các mảng Web – Mobile – AI và hoàn thành dự án đúng thời hạn với chất lượng tốt nhất.

5.4 Công nghệ sử dụng

5.4.1 Frontend

5.4.2 Backend

5.4.3 Database

5.4.4 AI Service

5.5 Xây dựng các module chức năng

5.5.1 Quản lý người dùng

5.5.2 Quản lý phòng trọ

5.5.3 Tìm kiếm và lọc

5.5.4 Đặt lịch xem phòng

5.5.5 Hệ thống đánh giá

5.6 Xây dựng module AI

5.6.1 AI Agent hỗ trợ tìm kiếm nhà trọ

5.6.2 AI đánh giá giá bất động sản

5.6.3 AI tạo nội dung bài đăng

5.6.4 Hệ thống gợi ý cá nhân hóa

5.6.5 AI kiểm duyệt nội dung bài đăng

5.7 Triển khai hệ thống

5.7.1 R2

5.7.2 MySQL

từ khám phá, tìm kiếm, xem chi tiết, so sánh, lưu tin cho tới liên hệ trực tiếp với chủ trọ/môi giới. Phần này trình bày các kết quả chính đã đạt được trên ứng dụng web. Các tính năng AI tích hợp trong ứng dụng (trợ lý hội thoại, gợi ý cá nhân hóa...) được trình bày chi tiết ở phần kết quả triển khai các tính năng AI; ở đây chỉ mô tả phần tích hợp và trải nghiệm phía giao diện.

5.9.1 Trang chủ và khám phá tin đăng

Trang chủ đóng vai trò điểm vào, tổng hợp nhiều khối nội dung giúp người dùng nhanh chóng định hướng: thanh tìm kiếm với bộ lọc nhanh theo địa điểm, loại hình và khoảng giá; khối thống kê tin đăng theo các tỉnh thành tiêu biểu (Hà Nội, Hồ Chí Minh, Đà Nẵng...); khối các loại hình cho thuê phổ biến; các nhóm tin nổi bật phân theo hạng VIP; khối tin gợi ý cá nhân hóa cho người dùng đã đăng nhập; và khối tin tức mới nhất. Các số liệu thống kê được lưu đệm theo ngày để giảm tải cho máy chủ và bảo đảm trang chủ hiển thị tức thì.

5.9.2 Tìm kiếm và lọc tin đăng

Trang danh sách tin đăng cung cấp khả năng tìm kiếm và lọc đa tiêu chí, gồm: khoảng giá, diện tích, loại hình (danh mục), vị trí (tỉnh/quận/phường), số phòng ngủ – phòng tắm, tiện ích, các loại chi phí (điện, nước, internet, phí dịch vụ), hướng nhà và sắp xếp theo nhiều tiêu chí. Toàn bộ điều kiện lọc được đồng bộ vào tham số trên thanh địa chỉ (URL), nhờ đó người dùng có thể chia sẻ hoặc lưu lại một kết quả tìm kiếm cụ thể, và trạng thái lọc được khôi phục khi mở lại đường dẫn. Kết quả được phân trang và hỗ trợ bố cục hai cột (danh sách kèm bộ lọc bên cạnh) trên màn hình lớn.

5.9.3 Trang chi tiết tin đăng

Trang chi tiết được kết xuất phía máy chủ (SSR) nhằm tối ưu SEO và tốc độ hiển thị lần đầu. Trang trình bày đầy đủ thông tin của một tin đăng: thư viện ảnh/video, tiêu đề và giá, danh sách tiện ích, mô tả chi tiết, biểu đồ lịch sử giá, bản đồ vị trí, và thẻ liên hệ chủ trọ/môi giới với chức năng hiện số điện thoại (có ghi nhận lượt bấm) cùng liên kết nhắn tin nhanh. Ở cuối trang còn có khối tin tương tự và khối tin vừa xem để gợi mở lựa chọn cho người dùng. Các thành phần nặng như bản đồ, biểu đồ và danh sách tin tương tự được tải động kèm khung chờ (skeleton) để không làm chậm phần nội dung chính.

5.9.4 Bản đồ tin đăng

Bên cạnh chế độ danh sách, người dùng có thể duyệt tin trên bản đồ (tích hợp Google Maps). Hệ thống định vị người dùng (hoặc dùng vị trí mặc định), gom nhóm các điểm tin gần nhau (marker clustering) để hiển thị gọn gàng, và đồng bộ danh sách tin trong khung nhìn hiện tại với một bảng bên cạnh. Khi người dùng di chuyển hoặc thu phóng bản đồ, hệ thống chỉ tải bổ sung tin cho vùng mới và lưu đệm các vùng đã tải để tránh gọi lại API không cần thiết, giúp thao tác trên bản đồ mượt mà.

5.9.5 So sánh tin đăng

Người dùng có thể chọn tối đa ba tin đăng để so sánh song song theo các thuộc tính quan trọng (giá, diện tích, số phòng, tiện ích, địa chỉ, các loại chi phí, hạng tin...). Danh sách so sánh được lưu tạm trong phiên làm việc của trình duyệt, hiển thị dưới dạng bảng đối chiếu trực quan giúp người dùng ra quyết định nhanh hơn.

5.9.6 Lưu tin và theo dõi người đăng

Đối với người dùng đã đăng nhập, hệ thống cho phép *lưu tin yêu thích* và quản lý chúng trong một trang riêng có phân trang. Ngoài ra, người dùng có thể *theo dõi* các chủ trọ/môi giới và xem bảng tin tổng hợp các tin đăng từ những người mình theo dõi, với giao diện chia thành hai thẻ “danh sách người theo dõi” và “bảng tin”. Đây là các tính năng tạo nên trải nghiệm cá nhân hóa và khuyến khích người dùng quay lại.

5.9.7 Trợ lý hội thoại AI trên giao diện

Ứng dụng tích hợp một widget trợ lý AI dạng nút nổi, có thể mở rộng thành cửa sổ hội thoại trên màn hình lớn hoặc toàn màn hình trên thiết bị di động. Về phía giao diện, widget nhận câu trả lời theo luồng thời gian thực (hiển thị dần từng đoạn kèm trạng thái xử lý), cho phép người dùng bấm vào các tin đăng do AI gợi ý để mở ngay hộp thoại chi tiết, ghi nhớ hội thoại trong phiên và hỗ trợ hủy luồng đang chạy. Năng lực xử lý ngôn ngữ và tìm kiếm phía sau của trợ lý được trình bày ở phần kết quả AI.

5.9.8 Đăng nhập và xác thực

Hệ thống cung cấp hai phương thức đăng nhập thuận tiện, không mật khẩu: *liên kết ma thuật* (magic link) gửi qua email và *đăng nhập bằng Google* (OAuth). Hộp thoại đăng nhập được thiết kế để bật lên ngay tại chỗ khi người dùng truy cập một chức năng yêu cầu xác thực, và sau khi đăng nhập thành công sẽ đưa người dùng trở lại đúng trang đang thao tác nhờ cơ chế `returnUrl`. Phiên đăng nhập được duy trì an toàn bằng cặp `access/refresh token` và tự động làm mới.

5.9.9 Tin tức và thông báo

Chuyên mục tin tức cung cấp danh sách bài viết có lọc theo chuyên mục, tìm kiếm theo từ khóa, bài nổi bật, khối “xem nhiều nhất” và trang chi tiết bài viết (kết xuất SSR). Bên cạnh đó, người dùng đã đăng nhập nhận được *thông báo thời gian thực* qua WebSocket (kèm hồi vòng dự phòng) về các sự kiện liên quan như tin được duyệt hay có tương tác, cùng chỉ báo số thông báo chưa đọc.

5.9.10 Trải nghiệm đa thiết bị và đa ngôn ngữ

Toàn bộ các trang trên đều được thiết kế đáp ứng cho cả máy tính và thiết bị di động, hỗ trợ song ngữ Việt – Anh và chuyển đổi tức thời. Các yếu tố như chỉ báo tiến trình tải trang, khung chờ khi tải dữ liệu, trạng thái rỗng có hướng dẫn rõ ràng và chuyển đổi đơn vị tiền tệ theo ngôn ngữ góp phần tạo nên trải nghiệm liền mạch và chuyên nghiệp cho người thuê.

5.10 Kết quả giao diện dành cho quản trị viên

5.10.1 Thống kê & Giám sát Hệ thống

5.10.2 Quản lý Người dùng & Phân quyền

5.10.3 Quản lý tin đăng và kiểm duyệt nội dung

5.10.4 Quản lý doanh thu và thanh toán

Chương 6

Kiểm thử và đánh giá hệ thống

6.1 Kế hoạch kiểm thử

6.2 Kiểm thử chức năng

6.3 Kiểm thử API

6.4 Kiểm thử bảo mật

6.5 Kiểm thử hiệu năng

6.6 Đánh giá chức năng AI

6.6.1 Độ chính xác của gợi ý phòng

6.6.2 Chất lượng phản hồi chatbot

6.6.3 Khảo sát mức độ hài lòng người dùng

6.7 So sánh với các hệ thống hiện có

Chương 7

Kết luận và hướng phát triển

7.1 Kết quả đạt được

7.2 Kiến thức và kinh nghiệm thu nhận được

7.2.1 Về mặt kỹ thuật

7.2.2 Về kỹ năng làm việc nhóm và lập kế hoạch

7.2.3 Một số kiến thức và kinh nghiệm khác

7.3 Hạn chế của hệ thống

7.4 Hướng phát triển

Tài liệu tham khảo

Tiếng Anh

- [1] Batdongsan.com.vn. “Batdongsan.com.vn - kênh thông tin bất động sản,” Accessed: Jun. 15, 2026. [Online]. Available: <https://batdongsan.com.vn/>
- [2] Phongtro123. “Phongtro123.com - kênh thông tin phòng trọ, nhà trọ,” Accessed: Jun. 15, 2026. [Online]. Available: <https://phongtro123.com/>
- [3] Chợ Tốt. “Nhà tốt - mua bán, cho thuê nhà đất,” Accessed: Jun. 15, 2026. [Online]. Available: <https://www.nhatot.com/>
- [4] Zillow Group, Inc. “Zillow - real estate, apartments, mortgages & home values,” Accessed: Jun. 15, 2026. [Online]. Available: <https://www.zillow.com/>
- [5] Meta Platforms, Inc. “React documentation - the library for web and native user interfaces,” Accessed: Oct. 15, 2025. [Online]. Available: <https://react.dev/>
- [6] Vercel, Inc. “Next.js documentation - the react framework for the web,” Accessed: Jun. 15, 2026. [Online]. Available: <https://nextjs.org/docs>